

Delphi App Translation Studio Wizard-to-Runtime Complete Procedure

A practical route from first scan to a tested running application

Version 2 | Last changed 7 October 2026

Contents

What this procedure covers....3
Prepare the Delphi project....3
Run the first translation....3
Connect the runtime in RAD Studio....4
Confirm packs beside the executable....5
Add more languages later....5
Recover if needed....6

What this procedure covers

This is the complete practical route from a saved Delphi project to a translated application you can run. The Setup Wizard handles the first target language. Maintenance Studio handles later additions and updates. The final check happens in the actual VCL or FireMonkey application, not merely in a saved catalog.

Source language means the language already written in the project. Target language means an added display language. A catalog is your editable translation record. A runtime pack is a validated JSON file read by the application. Saving a catalog and exporting a pack are separate actions.

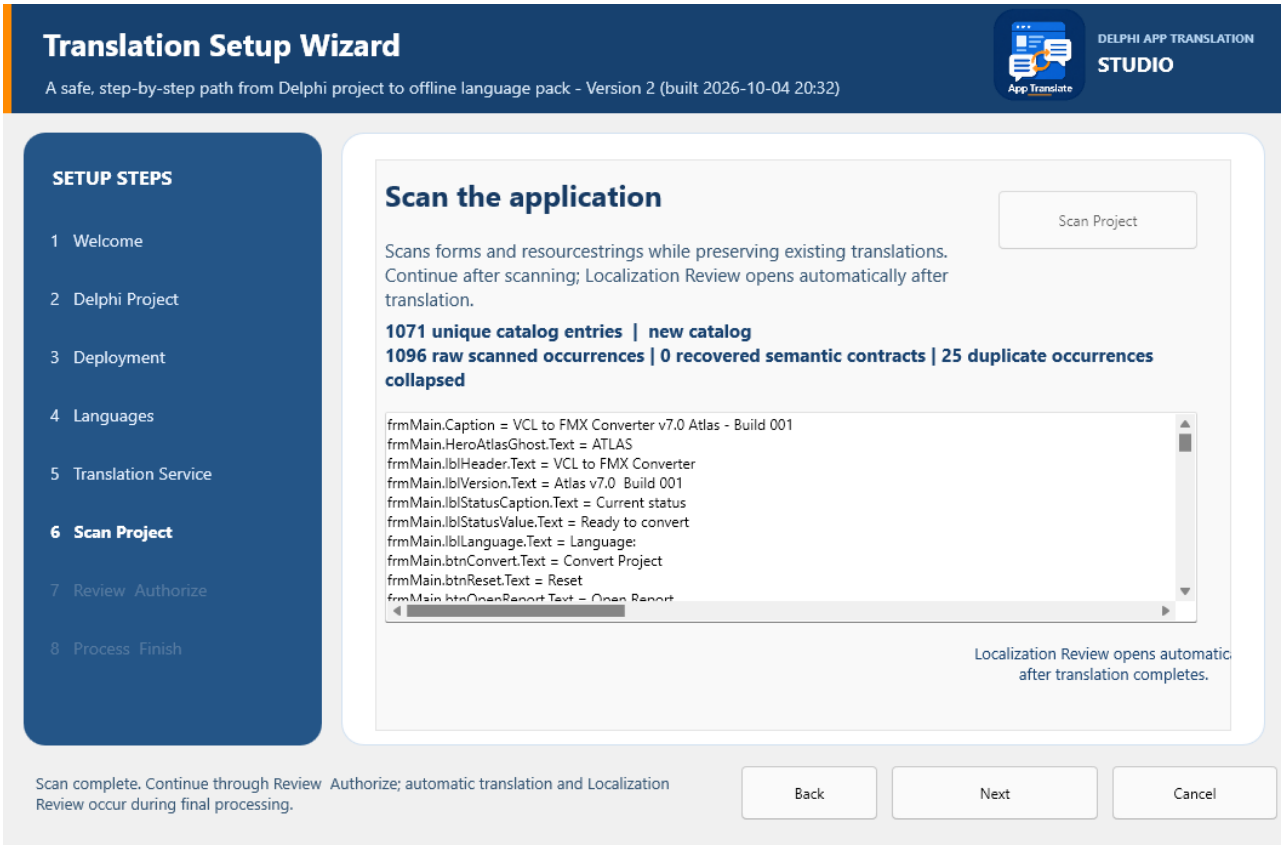
Prepare the Delphi project

1. In RAD Studio, save the project and forms. Close the running copy of the application you intend to rebuild or update.
2. Identify the .dproj or .dpr file and the language in its saved text.
3. Know which build outputs or separate installed/portable folders you need to test. An extra application folder is optional.
4. Have a DeepL or Google Cloud Translation key ready for automatic translation. Provider requests may incur charges.

Run the first translation

From the Studio start screen choose Run Setup Wizard. On Delphi Project, open the saved project and choose Automatic (Recommended), Create New Translation, or Update Existing Translation. On Deployment, review detected executable folders and add separate destinations only where needed; executable replacement is a separate authorization. On Languages, select the real source language and one different target. On Translation Service, choose a provider, enter or use a saved key, and test the connection.

Choose Scan Project. Read the sample entries and the unique/raw counts. If the wrong application or language appears, go back now. Scanning reads saved forms and source strings but does not rewrite them.



A completed scan is the checkpoint for project identity and source-language text before final processing.

On Review and Authorize, verify the project, service, languages, and deployment destinations. Confirm that target files are saved and the running application is closed. Choose Begin Final Processing. The Studio creates and checks its required ZIP Safety Backup first. Larger projects may take several minutes. Next it prepares draft translations and opens Localization Review. Inspect the text and proposed layout, then close Review so validation, export, dependency preparation, builds, and deployment can continue.

Wait for successful completion. The finish page offers Add more languages, Open Maintenance, Return to start, and manual Deploy to App Folder for a separate location. Manual deployment stays unavailable until processing succeeds. Note the destination shown in the result rather than assuming the executable you normally launch is the one that was updated.

Connect the runtime in RAD Studio

The Wizard prepares a project-local dependency and component kit, but does not insert controls into your forms or permanently edit project settings. On Integration, choose Prepare / Update Dependencies if the kit needs refreshing. Add $(PROJECTDIR)\dependencies\DelphiAppTranslation\source to the Delphi Compiler Search path in Project > Options > Building for each platform and configuration you ship. Use Show Design BPL, then Component > Install Packages > Add to install that exact design package in RAD Studio.

In the form designer, place one manager and one connected selector on the application's primary form. VCL uses TDATVCLLanguageManager and TDATVCLLanguageComboBox. FireMonkey uses

TDATFMXLanguageManager and TDATFMXLanguageComboBox. Set ApplicationId, SourceLanguage, and LanguagesFolder (Localization\Languages) on the manager. Set the selector's LanguageManager property to the manager. These properties can be inspected and changed in the Object Inspector. Save the form and project.

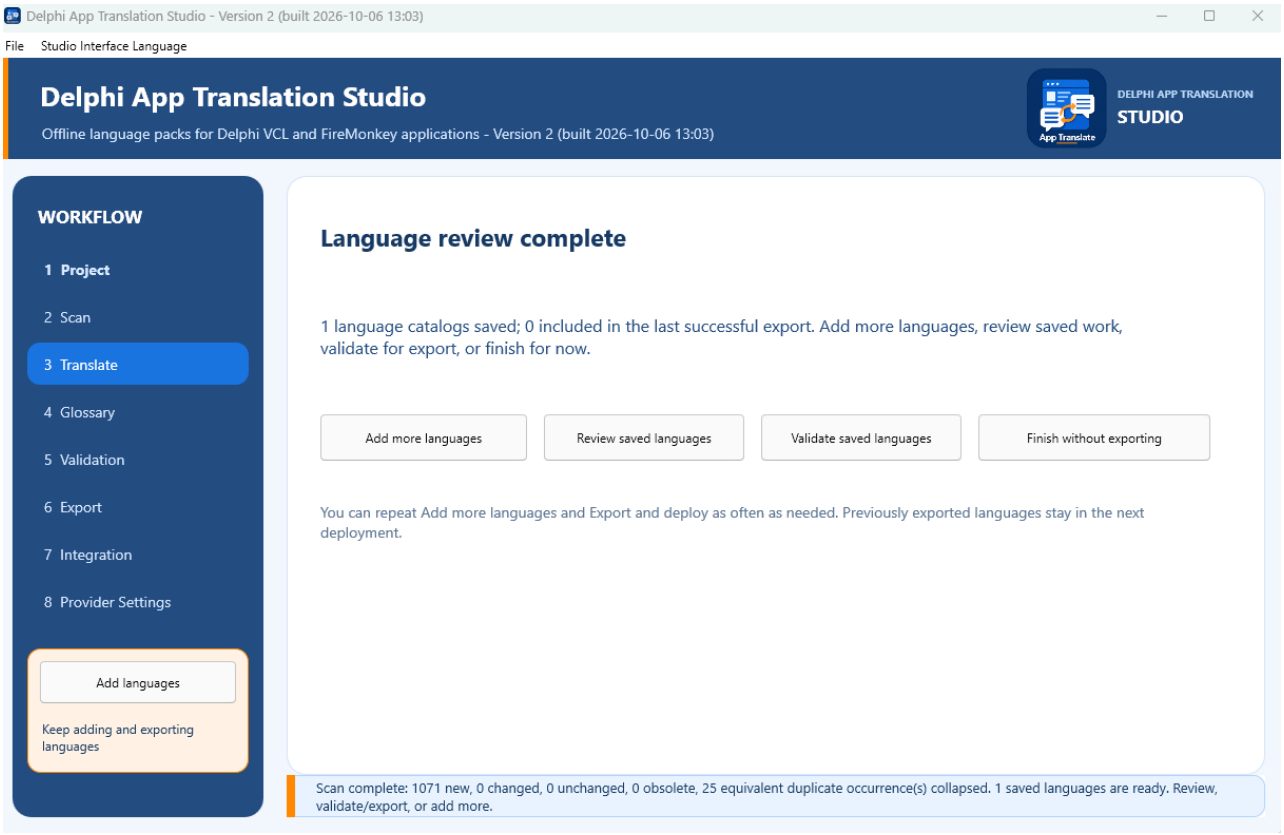
Confirm packs beside the executable

Find the exact executable you will run. Its folder should contain a Localization\Languages subfolder with the source pack and the new target pack. The project-level Localization\Languages copy supports later builds; it is not a substitute for packs beside an independently copied executable. Build and run each platform/configuration you distribute.

Switch from the source language to the target and back. Close and reopen the application to check that the selection persists. Inspect forms, menus, dialogs, dynamic messages, long labels, keyboard shortcuts, and placeholders such as %s. Try a right-to-left language in the running form if you plan to ship one. If text needs correction, return to the Studio catalog, save, validate, export, and retest.

Add more languages later

Open Maintenance Studio, then open the same Delphi project. A scan is useful if saved source text changed; it does not require you to choose all previous languages again. Go to Translate and choose Add languages. Select one or several new targets and choose Translate Selected. Watch the inline progress and batch results; several requests can take a few minutes. Existing catalogs remain saved.



After reviewing a language, choose Add more languages, Review saved languages, Validate saved languages, or Finish without exporting.

Inspect and save each new catalog. Choose Validate saved languages. Resolve errors in each affected language and validate again. Warnings call for review but do not automatically prevent export. A clean language can be exported while other languages remain drafts. On Export, check the ready set and every folder, then choose Export and deploy. The new export includes previously exported languages that remain in the validated set; you do not reselect them on the Add languages screen.

Recover if needed

From the start screen choose Recover from Safety Backup. Select the Studio ZIP, inspect the preview, and choose a new empty destination. The recovery action does not overwrite the original project. Open the restored copy in RAD Studio only after confirming it is the intended snapshot. For an ordinary translation edit, use the saved catalog and a new export instead of project recovery.